

RecPilot

An ML agent that researches on its own, and catches itself when it's wrong.

The Manual ML Improvement Loop

Every ML improvement follows the same five stage loop, done by hand. Two of the five stages are pure code, which makes them a natural target for an agent.

01

Read the problem

The engineer studies the data and the target metric.

02

Inspect data

Exploratory analysis surfaces what the data actually looks like.

03

Engineer features

This stage is written in code, so it can be automated.

04

Train and tune

This stage is also written in code, so it can be automated too.

05

Evaluate

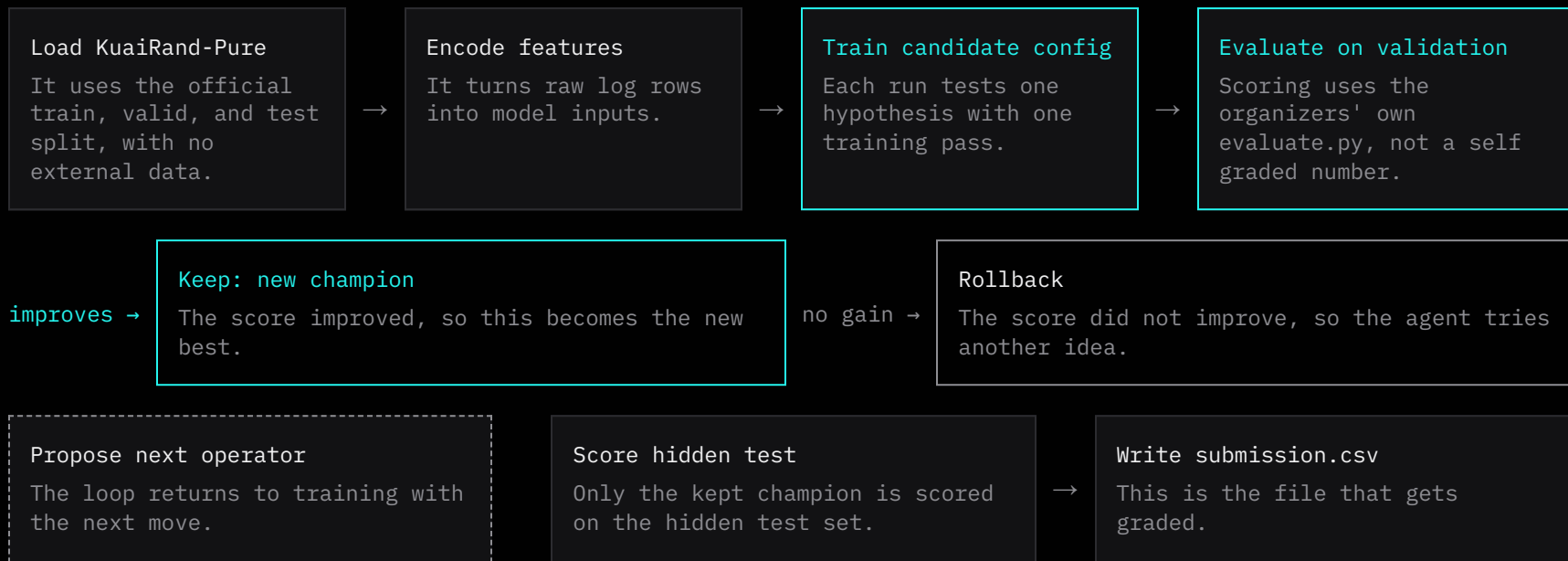
The engineer checks the score and decides what to change next.

The engineer reflects, revises, and returns to stage 02.

The challenge is to build an agent that runs this loop unattended on KuaiRand-Pure, beats a fixed baseline, and produces a full audit trail.

System Architecture

The agent runs its own experiment loop. It proposes a change, trains it, evaluates it, and keeps or rolls it back, with no human involved.



Champion vs Challenger

Every iteration is a contest. The current best config, the champion, competes against a new idea, the challenger. Only real improvements survive.

Propose a move

The move is picked from a fixed list, not a free for all.



Train the challenger

One move gets one training run.



Score on validation

Scoring uses the same official `evaluate.py` as every other slide.



Beats the
champion?

yes: new champion

This only happens
when the gain is
real, not noise.

no: rollback

The reason is logged
and the move goes on
cooldown.

THE RULEBOOK OF MOVES

Change the loss function

Add user-history features

Tune a hyperparameter

Blend two model types

Bag multiple random seeds

BACKUP SLIDE, PULL IT UP ONLY IF ASKED

One Iteration, Under the Hood

Agent loop

Planner

Subprocess, isolated and timed out

evaluate.py

Run log

01	Loop to planner	The loop asks the planner to propose the next move.
02	Planner to loop	The planner returns an operator and a hypothesis.
03	Loop to subprocess	The candidate is trained in an isolated subprocess.
04	Subprocess to evaluate.py	The subprocess scores the candidate on validation.
05	evaluate.py to subprocess	The official script returns GAUC and nDCG@5.
06	Subprocess to loop	The subprocess returns a result, a timeout, or an error.
07	Loop to run log	The loop records the hypothesis, config diff, score, and decision.
08	Loop to itself	If it beats the champion the loop keeps it, otherwise it rolls back.

Validation Results

METRIC	BASELINE	RECPILOT	DELTA
GAUC	0.6674	0.6718	+0.0044
nDCG@5	0.5357	0.5384	+0.0027
Primary (mean)	0.6016	0.6051	+0.0035
Validation delta	—	—	+0.0035

REFERENCE POINTS ON THE TEST SPLIT, FOR SCALE

Random · 0.4753

Item popularity · 0.5715

Official FM baseline · 0.5946

Label oracle · 0.8645



Practical Impact and Next Steps

This was built on Kuaishou's short video feed data, the same problem shape as TikTok's own For You ranking. The next step is to scale it up and let it run wherever engineers are currently doing this work by hand.

WHY IT MATTERS BEYOND THE BENCHMARK

Same problem, real stakes. KuaiRand-Pure is a short video feed ranking task, the same shape of problem behind TikTok's own recommendation systems.

Cuts the expensive part. Hyperparameter sweeps and feature tests that used to take an engineer days now run unattended, on CPU, for a few dollars in LLM calls.

Catches its own mistakes before they ship. The same leak guard that caught a label leaking model here matters even more at production scale.

NEXT STEPS

Scale to the larger KuaiRand benchmarks, 1k and 27k. The harness and operator catalog already support them.

Turn on the sequence interest and DeepFM plus DIN models already built into the model zoo. They were deprioritized here because user histories were short.

Swap the target label. Pointed at watch time, session length, or retention instead of `long_view`, the same agent can optimize a different product goal without a rewrite.

Thank you

github.com/aryand43-recpilot